Class Diagram for Library Management System in StarUML

A **Class Diagram** shows the classes of the system, their **attributes, methods, and relationships**.

For a Library Management System, we can use these main classes:

- Library
- Book
- Member
- Librarian
- Issue
- Fine
- Author

Step 1: Open StarUML

1. Open **StarUML**.
2. Select **File → New**.
3. In the **Model Explorer**, locate **Model**.
4. Right-click **Model**.
5. Select: **Add Diagram → Class Diagram**

A blank class diagram will appear.

Step 2: Create the Book Class

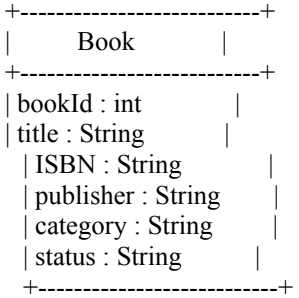
1. Select **Class** from the Toolbox.
2. Click on the diagram.
3. Rename the class to **Book**.
4. Right-click the Book class.
5. Select **Add → Attribute**.
6. Add these attributes:

bookId : int
title : String
ISBN : String
publisher : String
category : String
status : String

7. Add operations/methods:

searchBook()
issueBook()
returnBook()
updateStatus()

The class will look approximately like:



| searchBook() |
| issueBook() |
| returnBook() |
| updateStatus() |
+---------------------------+

Step 3: Create Relationships
This is the **most important part** of the Class Diagram.

In StarUML, relationships are available in the **Toolbox**.

You can use:

- Association
- Aggregation
- Composition
- Generalization

3.1 Member — Issue

A member can have many issue records.

Use **Association**.

Member 1 ———————— 0..* Issue
Meaning:

One member can have zero or many issue records.

In StarUML:

1. Select **Association**.
2. Click Member.
3. Drag to Issue.
4. Add multiplicity 1 near Member.
5. Add multiplicity 0..* near Issue.

Step 3.2 : Book — Issue
A book can appear in many issue records over its lifetime.

Book 1 ———————— 0..* Issue
Meaning:

One book can have zero or many issue records.

Create this using **Association**.

Step 3.3 : Issue — Fine
An issue may have a fine if the book is returned late.

Issue 1 ———————— 0..1 Fine
Meaning:

One issue may have zero or one fine.

Use **Association**.

Step 3.4 : Book — Author
One author can write multiple books, and a book may have multiple authors.

Therefore:

Author 0..* ——————— 0..* Book
This represents a **many-to-many relationship**.

Use **Association**.

Step 3.5 : Library — Book
A library contains books.

You can use **Aggregation**:

Library ◇——————— 0..* Book
The **hollow diamond** represents aggregation.

In StarUML:

1. Select **Aggregation**.
2. Click Library.
3. Drag to Book.
4. The diamond should be on the **Library** side.

Step 3.6 : Library — Member
A library has many members.

Library 1 ◇——————— 0..* Member
Use **Aggregation**.

Step 3.7 : Librarian — Book
A librarian manages books.

Librarian 1 ——————— 0..* Book
Use **Association**.

Step 4 : Add Multiplicity
Multiplicity tells us **how many objects participate in a relationship**.

Common multiplicities:

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many

For our Library Management System:

Member 1 ——————— 0..* Issue

Book 1 ——————— 0..* Issue

Issue 1 ——————— 0..1 Fine

Author 0..* ——— 0..* Book

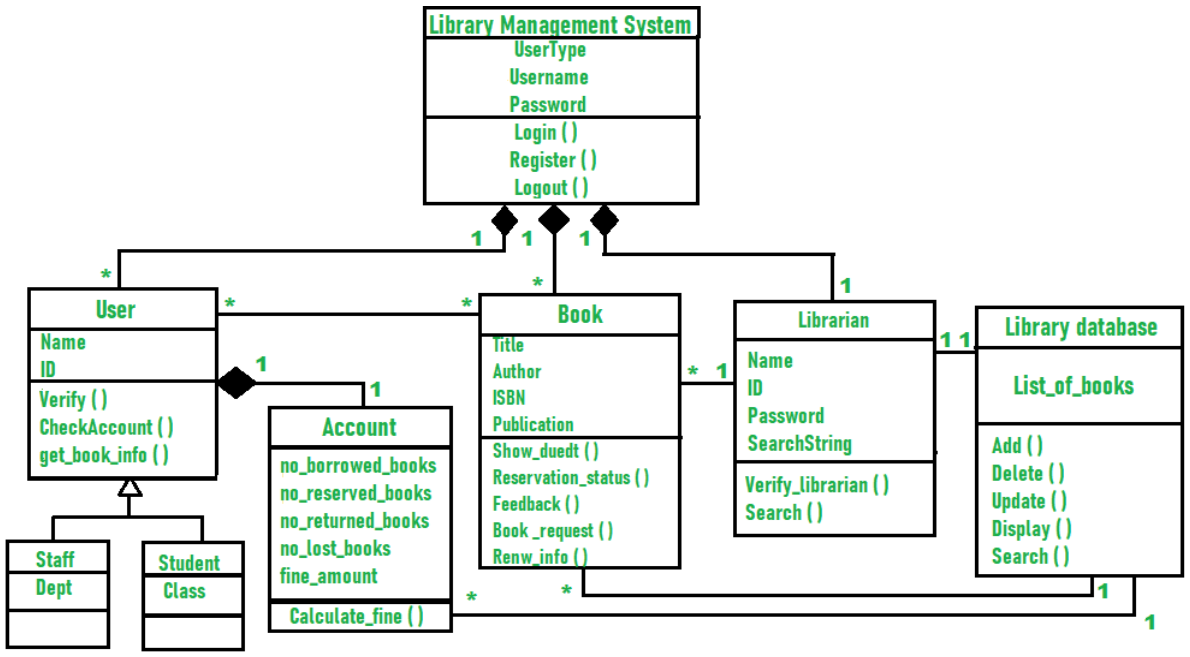
Library 1 ◇——————— 0..* Book

Library 1 ◇——————— 0..* Member

: How to Add Multiplicity in StarUML
After creating an association:

1. Click the relationship line.
2. Select the appropriate end of the relationship.
3. In the **Property Editor**, find **Multiplicity**.
4. Enter:
    - 1
    - 0..*
    - 0..1, etc.

Repeat for the other end.



CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM

Use case diagram for Library management system

Here is a step-by-step method to draw a Use Case Diagram in StarUML, using a Library Management System as an example.

1. Open StarUML

1. Open **StarUML**.
2. Create a new project:
    - **File → New**
3. In the **Model Explorer**, right-click **Model**.
4. Select:
   Add Diagram → Use Case Diagram

You will now get a blank Use Case Diagram canvas.

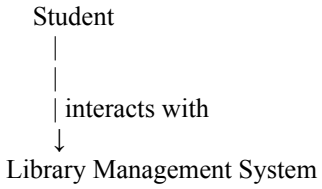
2. Identify the Actors

An **actor** represents a person or external system that interacts with your software.

For a Library Management System, common actors are:

- **Student/Member**
- **Librarian**
- **Admin**

Example:



3. Add an Actor in StarUML

From the **Toolbox** on the left:

1. Find **Actor**.
2. Click **Actor**.
3. Click anywhere on the diagram.
4. An actor will appear.
5. Double-click its name.
6. Rename it to **Student**.

Repeat the same process for:

- Librarian
- Admin

4. Create the System Boundary

The **system boundary** shows which functions belong to the Library Management System.

From the Toolbox:

1. Select **Subject**.
2. Draw a large rectangle around the use cases.
3. Rename it:

Library Management System

The actors should remain **outside** the rectangle.

5. Add Use Cases

A **use case** represents a function provided by the system.

For the Library Management System, you can use:

- Login
- Search Book
- Issue Book
- Return Book

- Renew Book
- Pay Fine
- Manage Books
- Manage Members
- Generate Reports

In StarUML:

1. Select **Use Case** from the Toolbox.
2. Click inside the system boundary.
3. Enter the use-case name.
4. Repeat for all required functions.

A use case is represented by an **oval**.

Example:

(Search Book)

○

6. Connect Actors to Use Cases

Use an **Association** to show that an actor interacts with a use case.

For example:

Student ———————— (Search Book)
Student ———————— (Issue Book)
Student ———————— (Return Book)
Librarian ———————— (Manage Books)
Librarian ———————— (Manage Members)
Admin ———————————— (Generate Reports)

In StarUML:

1. Select **Association** from the Toolbox.
2. Click the actor.
3. Drag to the appropriate use case.
4. Release the mouse.

7. Add <<include>> Relationship

Use **include** when one use case **always requires another use case**.

For example:

(Issue Book) ——<<include>>——> (Login)

This means the user must log in before issuing a book.

In StarUML:

1. Select **Include** from the Toolbox.
2. Click the main use case.
3. Drag to the included use case.

4. StarUML creates a dashed arrow with <<include>>.

8. Add <<extend>> Relationship

Use **extend** when an additional function happens **only under certain conditions**.

Example:

(Return Book)
↑
|
<<extend>>
|
(Pay Fine)

If a returned book is overdue, the **Pay Fine** function may be performed.

In StarUML:

1. Select **Extend**.
2. Click the extending use case.
3. Drag toward the base use case.

